



Řešení úkolu z kursu Teoretická informatika
Úkol 2 – TIN 2025/2026Z

Brno, 7. prosince 2025

David Kvaček
xkvace00@stud.fit.vutbr.cz

Úloha 1

Uvažujme jazyky L_1 a L_2 :

- $L_1 = \{\langle M \rangle \# \langle G \rangle \mid M \text{ je TS a } G \text{ je gramatika typu 2, t.ž. } \exists w \in \{0, 1\}^* : w \in L(M) \wedge w \notin L(G)\},$
- $L_2 = \{\langle M \rangle \# \langle G \rangle \mid M \text{ je TS a } G \text{ je gramatika typu 2, t.ž. } \exists w \in \{0, 1\}^* : w \notin L(M) \wedge w \in L(G)\}.$

Klasifikace a důkaz (ne-)příslušnosti do tříd jazyka L_1

Příslušnost jazyka L_1 do třídy RE (rekurzivně vyčíslitelné) lze dokázat sestrojením NTS (nedeterministického Turingova stroje) T_1 , který tento jazyk akceptuje:

- NTS T_1 ověří, zda jeho vstup odpovídá validnímu kódu $\langle M \rangle \# \langle G \rangle$, pokud ne, odmítne,
- NTS T_1 nedeterministicky vygeneruje řetězec w ,
- NTS T_1 ověří, zda platí $w \notin L(G)$, pokud ne, odmítne,
- NTS T_1 spustí simulaci TS M na řetězci w ,
- NTS T_1 akceptuje vstup, pokud TS M akceptuje řetězec w .

Nepříslušnost jazyka L_1 do třídy REC (rekurzivní) lze dokázat existencí redukce HP (Halting Problem, problém zastavení TS) na jazyk L_1 , tj. $HP \leq L_1$:

- požadovaná redukce je funkce $\sigma : \{0, 1, \#\}^* \rightarrow \{0, 1, \#\}^*$ definovaná jako:

$$\sigma(\langle M \rangle \# \langle w \rangle) = \langle M' \rangle \# \langle G \rangle$$

- zvolíme G , pevnou, bezkontextovou gramatiku typu 2 takovou, že její jazyk je neprázdný, ale není to celý jazyk, tj. $\{0\}^*$ s pravidly $S \rightarrow 0S \mid \epsilon$, a tedy existuje řetězec $w_0 \in \{0, 1\}^*$ takový, že $w_0 \notin L(G)$, např. $w_0 = 1$,
- pokud vstup $\langle M \rangle \# \langle w \rangle$ není korektní instance HP , tak funkce σ vrací kód TS M' takového, že $L(M') = \emptyset$ (tj. $\langle M' \rangle \# \langle G \rangle \notin L_1$), jinak σ vrací kód TS M' , který pracuje na jakémkoliv vstupu $w' \in \{0, 1\}^*$ následovně:
 - M' nejdříve spustí simulaci stroje M na vstupu w ,
 - pokud simulace cyklí, tak M' cyklí pro svůj vstup w' , a v tomto případě platí $L(M') = \emptyset$,
 - pokud simulace skončí, tak M' akceptuje svůj vstup w' , a v tomto případě platí $L(M') = \{0, 1\}^*$,
- tudíž pro M' platí:

$$\langle M \rangle \# \langle w \rangle \in HP \Rightarrow L(M') = \{0, 1\}^* \Rightarrow \exists w \in \{0, 1\}^* : w \in L(M') \wedge w \notin L(G) \Rightarrow \langle M' \rangle \# \langle G \rangle \in L_1$$

$$\langle M \rangle \# \langle w \rangle \notin HP \Rightarrow L(M') = \emptyset \Rightarrow \neg \exists w \in \{0, 1\}^* : w \in L(M') \wedge w \notin L(G) \Rightarrow \langle M' \rangle \# \langle G \rangle \notin L_1$$

$$\iff \langle M' \rangle \# \langle G \rangle \in L_1 \Rightarrow \langle M \rangle \# \langle w \rangle \in HP$$

$$\text{neboli } \langle M \rangle \# \langle w \rangle \in HP \iff \sigma(\langle M \rangle \# \langle w \rangle) \in L_1$$

Jazyk L_1 je RE , ale není REC , tj. $L_1 \in RE, L_1 \notin REC$.

Klasifikace a důkaz (ne-)příslušnosti do tříd jazyka L_2

Nepříslušnost jazyka L_2 do třídy REC lze dokázat existencí redukce $co-HP$ na jazyk L_2 , tj. $co-HP \leq L_2$:

- požadovaná redukce je funkce $\sigma : \{0, 1, \#\}^* \rightarrow \{0, 1, \#\}^*$ definovaná jako:

$$\sigma(\langle M \rangle \# \langle w \rangle) = \langle M' \rangle \# \langle G \rangle$$

- zvolíme G , pevnou, bezkontextovou gramatiku typu 2 takovou, že její jazyk je celý jazyk, tj. $\{0, 1\}^*$ s pravidly $S \rightarrow 0S \mid 1S \mid \epsilon$, a protože $w \in L(G)$ platí pro každé $w \in \{0, 1\}^*$, existenční podmínka se zjednoduší na $\exists w \in \{0, 1\}^* : w \notin L(M')$,
- pokud vstup $\langle M \rangle \# \langle w \rangle$ není korektní instance $co-HP$, tak funkce σ vrací kód TS M' takového, že $L(M') = \{0, 1\}^*$ (tj. $\langle M' \rangle \# \langle G \rangle \notin L_2$), jinak σ vrací kód TS M' , který pracuje na jakémkoliv vstupu $w' \in \{0, 1\}^*$ následovně:
 - M' nejdříve spustí simulaci stroje M na vstupu w ,
 - pokud simulace cyklí, tak M' cyklí pro svůj vstup w' , a v tomto případě platí $L(M') = \emptyset$,
 - pokud simulace skončí, tak M' akceptuje svůj vstup w' , a v tomto případě platí $L(M') = \{0, 1\}^*$,
- tudíž pro M' platí:

$$\langle M \rangle \# \langle w \rangle \in co-HP \Rightarrow L(M') = \emptyset \Rightarrow \exists w \in \{0, 1\}^* : w \notin L(M') \wedge w \in L(G) \Rightarrow \langle M' \rangle \# \langle G \rangle \in L_2$$

$$\langle M \rangle \# \langle w \rangle \notin co-HP \Rightarrow L(M') = \{0, 1\}^* \Rightarrow \neg \exists w \in \{0, 1\}^* : w \notin L(M') \wedge w \in L(G) \Rightarrow$$

$$\langle M' \rangle \# \langle G \rangle \notin L_2 \iff \langle M' \rangle \# \langle G \rangle \in L_2 \Rightarrow \langle M \rangle \# \langle w \rangle \in co-HP$$

$$\text{neboli } \langle M \rangle \# \langle w \rangle \in co-HP \iff \sigma(\langle M \rangle \# \langle w \rangle) \in L_2$$

Jazyk L_2 není REC , a protože platí $REC \subset RE$, není ani RE , tj. $L_2 \notin RE, L_2 \notin REC$.

Úloha 2

Důkaz, že $\text{ERR-COLORING} = \{(G, k, j) \mid \text{graf } G \text{ lze obarvit } k \text{ barvami s nejvýše } j \text{ chybami}\}$ (problém obarvení s chybami) je NP-úplný, lze provést tak, že ukážeme náležitost problému do třídy NP (nedeterministicky polynomiální) a rovněž je možné na něj převést jiný známý NP-úplný problém (tj. NP-těžký), v tomto případě $k\text{-COLORING} = \{(G, k) \mid \text{graf } G \text{ lze obarvit } k \text{ barvami}\}$ (Graph Coloring Problem, problém obarvení grafu).

ERR-COLORING $\in$ NP

NTS si nedeterministicky zvolí přiřazení barev jednotlivým vrcholům grafu (funkce $c : V \rightarrow \langle 1, k \rangle$) a v polynomiálním čase ověří, zda toto obarvení splňuje uvedenou podmínku, tj. počet hran spojujících vrcholy stejné barvy $\leq j$.

$k\text{-COLORING} \leq_p \text{ERR-COLORING}$

Zkonstruuujeme polynomiálně vyčíslitelnou funkci f , pro kterou platí:

$$A \in k\text{-COLORING} \Leftrightarrow f(A) \in \text{ERR-COLORING}$$

Funkce f pracuje následovně:

- nechť $A = (G, k)$ je instance problému $k\text{-COLORING}$, kde $G = (V, E)$ je graf a k je počet barev,
- funkce f vytvoří instanci pro problém ERR-COLORING $f(A) = (G', k', j')$, kde:
 - G' je graf, který vznikne přidáním úplného grafu K s počtem vrcholů $k+1$ jako samostatné komponenty k původnímu grafu G ,
 - $k' = k$ je počet barev,
 - $j' = 1$ je počet povolených chyb.

Je zřejmé, že platí:

$$G \text{ je obarvitelný } k \text{ barvami (bez chyb)} \Leftrightarrow G' \text{ je obarvitelný } k \text{ barvami s nejvýše } j' \text{ chybami}$$

$\Rightarrow$:

Pokud je graf G obarvitelný k barvami bez chyb, tak v rámci grafu G' obarvíme část G validním způsobem bez chyb a část odpovídající K obarvíme k barvami s jednou chybou, jelikož má tento úplný graf $k+1$ vrcholů a mezi každými dvěma vrcholy vede hrana, celkový počet chyb v grafu G' je tedy $0 + 1 = 1$ a protože $1 \leq j'$ (kde $j' = 1$), tak platí $f(A) \in \text{ERR-COLORING}$.

$\Leftarrow$:

Pokud je graf G' obarvitelný k barvami s nejvýše j' chybami, tak je počet chyb v G' součtem chyb v komponentě G a chyb v komponentě K , přičemž víme, že úplný graf K nelze obarvit k barvami bez chyb, tedy počet chyb v K je ≥ 1 a protože celkový počet chyb musí být maximálně 1, tak z toho plyne, že počet chyb v G musí být roven 0, tzn. původní graf G je obarvitelný k barvami bez chyb, platí tedy $A \in k\text{-COLORING}$.

Je zřejmé, že funkce f je polynomiálně vyčíslitelná a navíc pro ni platí výše uvedený vztah.

Úloha 3

Uvažujme teorii T se signaturou $\langle \{Cat_{/0}, Fish_{/0}, Trex_{/0}\}, \{eats_{/2}, =_{/2}\} \rangle$ ($=$ je standardní rovnost) se speciálními axiomy:

$$\forall x(x = Fish \vee x = Cat \vee x = Trex)$$

$$\forall x \forall y(eats(x, y) \leftrightarrow \neg eats(y, x))$$

$$\forall x(x = Trex \vee eats(Trex, x))$$

Rozhodnutí a zdůvodnění tvrzení o teorii T

- (a) Teorie T není bezesporná, protože neexistuje ani jeden model, ve kterém si axiomy neprotiřečí, což vychází z druhého axiomu $\forall x \forall y(eats(x, y) \leftrightarrow \neg eats(y, x))$ pro všechny dvojice prvků x, y z univerza, tedy v případě $x = y$ vychází $eats(x, x) \leftrightarrow \neg eats(x, x)$.
- (b) Teorie T je úplná, protože pokud je teorie T sporná, lze v ní dokázat každá formule i její negace a v rámci úplné teorie platí, že pro každou formuli φ lze dokázat buď φ , nebo $\neg \varphi$.
- (c) Teorie T je rozhodnutelná (tj. množina důsledků T je rozhodnutelná), protože každá formule ve sporné teorii je dokazatelná.

Rozhodnutí a zdůvodnění redukovatelnosti $T_{PA} \leq T$

Peanova aritmetika T_{PA} nemůže být redukovatelná na T (tj. nemůže existovat vyčíslitelná redukce f , která každou větu φ Peanovy aritmetiky převede na větu $f(\varphi)$ jazyka teorie T takovou, že $T_{PA} \models \varphi \iff T \models f(\varphi)$), tj. $T_{PA} \not\leq T$.

Teorie T je triviálně rozhodnutelná, protože je sporná, avšak Peanova aritmetika T_{PA} je nerozhodnutelná. Není možné provést redukci nerozhodnutelného problému (T_{PA}) na problém rozhodnutelný (T). Pokud by taková vyčíslitelná redukce f existovala, mohli bychom rozhodnout T_{PA} tak, že bychom problém převedli pomocí f a výsledek ověřili v T , to je však ve sporu s nerozhodnutelností T_{PA} .

Úloha 4

Uvažujme pole celých čísel proměnné délky, jehož datová struktura obsahuje proměnné (i) *arr*—ukazatel na začátek pole, (ii) *size*—velikost alokovaného prostoru, (iii) *used*—počet využitých položek od začátku. Pro práci s polem máme funkce *array_init*, *insert*, *sort*. Předpokládejme uniformní cenové kritérium, kde každý řádek má cenu 1 (s výjimkou volání funkce *sort*).

Analýza a zdůvodnění složitosti jednoho volání *insert* v nejhorším případě

Pro určení složitosti v nejhorším případě musíme uvažovat scénář s největším počtem provedených operací, k čemuž dojde ve chvíli, kdy je pole *array* plně obsazeno, tj. *array.size* = *array.used*. V tomto případě funkce *insert* provede následující kroky:

- vytvoří nové pole dvojnásobné velikosti a zkopíruje všechny prvky starého pole, složitost této operace je lineární $\mathcal{O}(n)$,
- vloží nový prvek na konec nového pole, složitost této operace je konstantní $\mathcal{O}(1)$,
- seřadí nové pole s již vloženým prvkem zavoláním funkce *sort* (Bubble Sort, bublinkové řazení) a v nejhorším případě při zcela opačném pořadí prvků je počet prohození maximální, složitost této operace je tedy kvadratická $\mathcal{O}(n^2)$.

Složitost jednoho volání funkce *insert* v nejhorším případě je určena voláním funkce *sort* kvůli nutnosti řazení pole po jeho zvětšení, tj. kvadratická $\mathcal{O}(n) + \mathcal{O}(1) + \mathcal{O}(n^2) = \mathcal{O}(n^2)$.

Analýza a zdůvodnění složitosti posloupnosti n operací *insert*

Amortizovaná složitost vyjadřuje průměrnou cenu posloupnosti n operací jako celku, přičemž stojí na základech pozorování, že drahé operace mohou nastat pouze jednou za delší dobu a tudíž se jejich cena amortizuje. Předpokládejme posloupnost n operací *insert*, kde se tyto operace dělí na dva typy – levné (vlození nového prvku, tj. *array.size* $\neq$ *array.used*) a drahé (vlození nového prvku, kopírování prvků do nového pole, řazení nového pole, tj. *array.size* = *array.used*).

Nechť n je celkový počet operací a k je aktuální počet prvků v poli. Při levné operaci *insert* se pouze vloží nový prvek s cenou 1. V rámci drahé operace *insert* se kopíruje k prvků, vloží se nový prvek a řadí se pole o velikosti $k + 1$ s celkovou cenou $(k + 1)^2 \approx k^2$.

Protože cena nejdražší operace roste kvadraticky (n^2), ale počet levných operací roste pouze lineárně (n), musí kredit záviset na celkovém počtu prvků n . Z pohledu $\mathcal{O}$ notace není podstatná konstanta, ale závislost na n , tedy lineární amortizovaná složitost $\mathcal{O}(n)$. Celková cena posloupnosti n operací je omezena shora $\leq \mathcal{O}(n^2)$.

Úloha 5

Nechť $G = (V, E)$ je orientovaný graf. Definujme sled jako $sled(v_1, v_n) \Leftrightarrow \exists$ posloupnost $v_1 e_1 v_2 e_2 \dots e_{n-1} v_n : \forall 0 < i < n : e_i = (v_i, v_{i+1})$ a $e_i \in E$. Uzly a hrany se mohou v posloupnosti opakovat. Pro graf $G = (V, E)$, uzly $v_s, v_f \in V$ a hranu $e \in E$ uvažujme problém existence sledu $sled(v_s, v_f)$, který obsahuje hranu e . Uvažujme funkci *check*, která prohledá všechny sledy do délky $2 * |V|$ a zjistí, zda takovýto sled existuje. Předpokládejme uniformní cenové kritérium, kde každý řádek má cenu 1.

Analýza časové složitosti funkce *check* v nejlepším případě

V nejlepším případě algoritmus nalezne hledaný sled okamžitě, aniž by musel prohledávat zbytek grafu. Předpokládejme, že hrana $e = (v_s, v_f)$ existuje, tj. začátek hrany je počáteční uzel a konec hrany je cílový uzel, a je hned prvním následníkem v seznamu $succ(v_s)$. V takovém případě proběhne pouze konstantní počet operací a časová složitost funkce *check* v nejlepším případě je tedy $\mathcal{O}(1)$.

Analýza časové složitosti funkce *check* v nejhorším případě

Funkce *check* provádí DFS (Depth-First Search, prohledávání do hloubky) všech možných sledů. Základním problémem je, že uvedený algoritmus neoznačuje navštívené uzly, ale pouze příznak průchodu hranou e v rámci aktuální cesty. Z toho plyne, že algoritmus může procházet stejné uzly opakovaně v různých větvích výpočtu až do hloubky $2 * |V|$.

V nejhorším případě hledaný sled neexistuje, nebo je nalezen až na konci. V každém kroku má uzel $|V|$ následníků a algoritmus se zanořuje až do hloubky $2 * |V|$. Časová složitost funkce *check* v nejhorším případě je tedy $\mathcal{O}(|V|^{2*|V|})$.

Návrh funkce *check_opt* s lepší asymptotickou složitostí v nejhorším případě

Problém lze vyřešit mnohem efektivněji dekompozicí na dva podproblémy, které jsou ekvivalentní, tj. zda existuje cesta z v_s do u (začátek hrany e) a cesta z v (konec hrany e) do v_f . Pokud obě tyto cesty existují, jejich spojením přes hranu $e = (u, v)$ vznikne platný sled. Návrh optimalizované funkce *check_opt* je tedy následující:

```
1 Function check_opt(( $V, E$ ),  $v_s, v_f, e$ ):
2   if  $e \notin E$  then
3     return false;
4    $start := \text{reach}((V, E), v_s, u);$ 
5    $end := \text{reach}((V, E), v, v_f);$ 
6   return  $start \wedge end$ ;

7 Function reach(( $V, E$ ),  $start, finish$ ):
8   if  $start = finish$  then
9     return true;
10   $queue := \text{empty\_queue}();$ 
11   $visited := \text{empty\_set}();$ 
12   $queue.enqueue(start);$ 
13   $visited.add(start);$ 
14  while  $queue \text{ not empty}$  do
15     $current := queue.dequeue();$ 
16    foreach  $w \in succ(current)$  do
17      if  $w = finish$  then
18        return true;
19      if  $w \notin visited$  then
20         $visited.add(w);$ 
21         $queue.enqueue(w);$ 
22  return false;
```

Analýza časové složitosti funkce *check_opt* v nejhorším případě

Funkce *check_opt* nejdříve ověřuje existenci hrany, což je v kontextu celého prohledávání zanedbatelné se složitostí $\mathcal{O}(1)$. První volání pomocné funkce *reach* standardním algoritmem BFS (Breadth-First Search, prohledávání do šířky) projde v nejhorším případě všechny vrcholy a všechny hrany dostupné z v_s . Díky množině *visited* se každý uzel a hrana zpracuje maximálně jednou s celkovou složitostí $\mathcal{O}(|V| + |E|)$. Druhé volání pomocné funkce *reach* má analogicky stejnou složitost $\mathcal{O}(|V| + |E|)$ jako první volání.

V nejhorším případě tedy dostáváme celkovou složitost $\mathcal{O}(|V| + |E|) + \mathcal{O}(|V| + |E|) = \mathcal{O}(|V| + |E|)$, kdy se sčítá složitost dvou nezávislých průchodů a funkce *check_opt* je ve srovnání s původní funkcí *check* lineární vzhledem k velikosti grafu $G = (V, E)$.